

# Function Pointers in C++ — Complete Guide

## 1. The Core Concept

A function pointer points to **code (machine instructions)** instead of data.

```
int x = 5;
int* p = &x;           // points to data in memory

void Print(int, char);
void (*fp)(int, char) = Print; // points to FUNCTION CODE
```

Memory:

```
+-----+
| Code Segment |
| +-----+ |
| | Print function | @ 0x401234
| +-----+ |
+-----+
| Stack |
| fp = 0x401234 | <- pointer holds this address
+-----+
```

## 2. Why Function Pointers Exist

```
Regular call:    Print(10, '#'); -> CALL 0x401234 (hardcoded by compiler)
Pointer call:    fp(10, '#');    -> CALL whatever fp holds RIGHT NOW (runtime)
```

Enables:

- Callbacks (pass a function to be called later)
- Customization without changing code
- Plug-in systems (load functions at runtime)
- Dispatch tables / state machines
- Event handlers

## 3. The Syntax

```

void (*printPtr)(int, char) = Print;
//   ^      ^      ^
//   |      |      +-- parameters
//   |      +-- pointer name (* makes it a pointer)
//   +-- return type

```

## Reading order

```

void   (*printPtr)   (int, char)
  ^         ^         ^
  |         |         |
+-- 3 -----+         |
          |         |
        +-- 1 -----+
                      |
                      +-- 2

```

1. printPtr is a POINTER
2. ... to a function taking (int, char)
3. ... returning void

## Parentheses around *\*name* are MANDATORY

```

void (*fp)(int);    // POINTER to function taking int, returning void
void *fp(int);      // DIFFERENT - function taking int, returning void*

```

## 4. Assigning and Calling

```

// Assignment - all equivalent
void (*fp)(int,char) = Print;    // implicit address
void (*fp)(int,char) = &Print;   // explicit address

// Calling - all four are valid and equivalent
Print(10, '#');                  // direct
(*Print)(10, '#');                // dereference (Print decays to pointer)
fp(10, '#');                      // through pointer (modern)
(*fp)(10, '#');                   // through pointer (classic)

```

Functions are special: name auto-decays to pointer, pointer auto-dereferences on call.

## 5. Cleaner Syntax with *typedef* / *using*

```
// Hard to read
void (*signal(int, void (*)(int)))(int);

// typedef (old style)
typedef void (*PrintFunc)(int, char);
PrintFunc fp = Print;

// using (modern, C++11) - same meaning, cleaner
using PrintFunc = void (*)(int, char);
PrintFunc fp = Print;
```

## 6. Function Pointers as Parameters (Callbacks)

The #1 real-world use:

```
void doTwice(void (*action)(int)) {
    action(1);
    action(2);
}

void print(int x) { cout << x; }
void square(int x) { cout << x*x; }

doTwice(print);    // SAME function, DIFFERENT behavior
doTwice(square);
```

### STL Algorithms Use This

```
bool descending(int a, int b) { return a > b; }
bool ascending (int a, int b) { return a < b; }

vector<int> v = {3, 1, 4, 1, 5};
sort(v.begin(), v.end(), descending);
sort(v.begin(), v.end(), ascending);
```

## 7. Arrays of Function Pointers (Dispatch Tables)

```
double add(double a, double b) { return a + b; }
double sub(double a, double b) { return a - b; }
double mul(double a, double b) { return a * b; }
double divv(double a, double b){ return a / b; }

double (*ops[4])(double, double) = { add, sub, mul, divv };
```

```
cout << ops[0](10, 3);    // add -> 13
cout << ops[2](10, 3);    // mul -> 30

int choice = 2;
cout << ops[choice](5, 5); // -> 25 (no if/switch needed!)
```

## 8. Function Pointer as Return Type

```
// Raw syntax - ugly
int (*getOperation(char op))(int, int) {
    if (op == '+') return add;
    return nullptr;
}

// With using - readable
using IntOp = int(*)(int, int);
IntOp getOperation(char op) {
    if (op == '+') return add;
    return nullptr;
}
```

## 9. Function Pointer vs `std::function` vs Lambda

| Feature             | Function Pointer            | <code>std::function</code>             | Lambda                      |
|---------------------|-----------------------------|----------------------------------------|-----------------------------|
| Syntax              | <code>void(*fp)(int)</code> | <code>function&lt;void(int)&gt;</code> | <code>[](int x){...}</code> |
| Captures variables? | No                          | Yes                                    | Yes                         |
| Type-safe           | Limited                     | Yes                                    | Yes                         |
| Performance         | Fastest                     | Slower (may heap-allocate)             | Often inlined               |
| Class members       | Special syntax              | Yes                                    | Yes                         |
| Header              | none                        | <code>&lt;functional&gt;</code>        | none                        |
| C compatible        | Yes                         | No                                     | No                          |

```
// Same task three ways
void (*fp)(int, char) = Print;           // function pointer
std::function<void(int, char)> sf = Print; // std::function
auto lam = [](int n, char c) { ... };     // lambda
```

## 10. Member Function Pointers — Different Beast

```

class Car {
public:
    void honk() { cout << "Beep!"; }
};

void (Car::*memFp)() = &Car::honk;
//      ^      ^      ^
//      class member ptr      MUST use &Class::method

Car c;
(c.*memFp)();      // call through object - note .*

Car* pc = &c;
(pc->*memFp)();      // call through pointer - note ->*

```

## 11. Real-World Uses

```

// atexit - register exit handler
atexit(cleanup);

// qsort - C-style sort
qsort(arr, n, sizeof(int), compareFunc);

// signal - OS signal handler
signal(SIGINT, handlerFunc);

// Event callbacks
button.onClick(handleClick);

// Thread entry point
pthread_create(&t, nullptr, threadFunc, args);

// Plugin systems
auto fn = (void(*)())dlsym(handle, "plugin_init");

// State machines
using StateFn = void(*)();
StateFn currentState = idleState;
currentState();

```

## 12. Cheat Sheet

```

DECLARATION:
return_type (*name)(params) = function;
void (*fp)(int, char) = Print;

```

**ASSIGNMENT:**

```
fp = Print;          // implicit address
fp = &Print;         // explicit (same)
```

**CALLING:**

```
fp(args);            // modern
(*fp)(args);         // classic
```

**ALIAS:**

```
using FnType = void(*)(int, char);
FnType fp = Print;
```

**CALLBACK PARAMETER:**

```
void run(void (*cb)(int)) { cb(5); }
```

**ARRAY OF POINTERS:**

```
void (*table[4])(int) = { fnA, fnB, fnC, fnD };
table[i](42);
```

**RULES:**

- Function name without () = address
- () around (\*name) is MANDATORY
- Signatures must match EXACTLY
- Class members need Class::\* syntax
- Cannot capture variables (use lambda/std::function)